The Journal of Open Source Software

PyBurgers: A Python testbed for direct and large-eddy simulation of stochastically forced one-dimensional Burgers turbulence

Jeremy A. Gibbs [1]

1 NOAA National Severe Storms Laboratory, Norman, OK, USA

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 11 June 2026
Published: unpublished

Summary

PyBurgers is an open-source Python solver for the one-dimensional stochastically forced Burgers equation (1D SBE). The software models complex fluid turbulence and energy dissipation in a simplified, one-dimensional environment. It supports both direct numerical simulation (DNS) and large-eddy simulation (LES) on a periodic domain. PyBurgers is intended as a lightweight testbed for studying turbulence phenomenology, evaluating subgrid-scale (SGS) turbulence closures, and prototyping new numerical methods.

The Burgers equation was originally conceived by Dutch scientist J.M. Burgers as one of the first attempts to arrive at a statistical theory of turbulent fluid motion (Burgers, 1939). His equation represents a very simplified model that describes the interaction of non-linear inertial terms and dissipation in the motion of a fluid, and it shares many characteristics with the Navier-Stokes equations: **advective non-linearity**, **diffusion**, **invariance**, and **conservation.** However, it is not ideal for the chaotic nature of turbulence because it can be integrated explicitly and is not sensitive to small changes in initial conditions.

A popular modification is the addition of a forcing term, the so-called stochastic term, that accounts for these neglected effects. This term perturbs the system with a stochastic process that is stationary in time and space. The resulting governing 1D SBE is given by

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2} + \eta(x,t), \tag{1}$$

where u is velocity, x is the spatial dimension, ν is kinematic viscosity, and η is the stochastic term. PyBurgers uses fractional Brownian motion (FBM):

$$\eta(x,t) = \sqrt{\frac{2D_0}{\Delta t}}\mathfrak{F}^{-1}\left\{|k|^{\beta/2}\hat{f}(k)\right\}, \tag{2}$$

where D_0 is noise amplitude, Δt is the model time step, $\mathfrak{F}^{-1}$ is the inverse Fourier transform, β is the spectral slope of the noise, and f is Gaussian random noise with mean $= 0$ and standard deviation $= \sqrt{N}$, where N is the number of grid points. The 1D SBE is a canonical minimal nonlinear partial differential equation that exhibits cascade-like energy transfer and shock-like dissipation structures. Equation (1) is widely used as a one-dimensional surrogate for the Navier-Stokes equations (Basu, 2009; e.g., Bec & Khanin, 2007).

The default spectral configuration uses Fourier collocation for spatial derivatives and 3/2-rule dealiasing for the nonlinear term. The same solver also supports second- and fourth-order finite-difference spatial operators (FD2 and FD4, respectively). Both spatial and temporal

The Journal of Open Source Software

discretizations are runtime-selectable from a JSON namelist, which is useful, e.g., for isolating discretization effects in SGS-model evaluation. Output is written to a NetCDF file.

PyBurgers is distributed under the MIT license at https://github.com/jeremygibbs/pyburgers with full documentation at https://docs.gibbs.science/pyburgers.

Statement of need

The Burgers equation is a well-established model problem in turbulence and LES research for several reasons, including: it is cheap enough to resolve on a single workstation because it is one-dimensional, and it is useful for testing SGS closures that target unresolved advective and dissipative processes because its non-linearity and forcing produce cascade-like transfer and intermittent shocks (Bec & Khanin, 2007; Love, 1980). Burgers turbulence has, therefore, become a recurring testbed for classical, implicit, and data-driven SGS closures (LaBryer et al., 2015; Li & Wang, 2016; Maulik & San, 2018; Subel et al., 2021).

However, available software occupies different niches despite this persistence. General PDE frameworks solve Burgers-type equations, while public teaching scripts often demonstrate a single deterministic case (e.g., Binder, 2021). Published Burgers LES studies also tend to emphasize a particular discretization, closure model, or data-driven experiment rather than a reusable package interface for stochastic DNS/LES turbulence workflows. PyBurgers addresses this narrower need: a maintained, documented, open-source Python package for Burgers-turbulence studies without requiring users to re-derive the spectral machinery or rebuild the DNS/LES workflow from scratch.

PyBurgers naturally targets three audiences:

1. **SGS-modeling researchers** who need a controlled, fast *a priori* / *a posteriori* environment in which to evaluate closures against filtered DNS or other sources of truth. Four classical SGS models ship with the code: constant-coefficient Smagorinsky (Smagorinsky, 1963), dynamic Smagorinsky (Germano et al., 1991), dynamic Wong–Lilly (Wong & Lilly, 1994), and a prognostic 1.5-order TKE closure after Deardorff (Deardorff, 1980). These serve as reference implementations for new closures, including data-driven ones (Subel et al., 2021), which are easily implemented by subclassing SGS.
2. **Numerical-methods researchers** who want a small benchmark on which to compare time integrators (Adams–Bashforth 2 (Bashforth & Adams, 1883), Adams–Moulton 2 predictor–corrector (Moulton, 1926), Williamson low-storage Runge-Kutta 3 (Williamson, 1980)) and spatial discretizations (spectral, and second- and fourth-order finite differences) without conflating those choices with modeling effects (Durran, 2010).
3. **Educators and students** who want a detailed example of a complete spectral turbulence solver at a scale that fits in a course module. Relevant PyBurgers features toward this end include, e.g., adaptive time-stepping, dealiasing, hyperviscosity, FFTW wisdom caching, NetCDF output.

State of the field

The relevant alternatives fall into complementary categories. The first comprises demonstration scripts that integrate the deterministic Burgers equation with a fixed scheme. This category is useful for teaching, but not designed as extensible turbulence packages (e.g., Binder, 2021). The second group consists of general PDE frameworks such as Dedalus (Burns et al., 2020), Nektar++ (Cantwell et al., 2015), and the SciML/DifferentialEquations.jl ecosystem (Rackauckas & Nie, 2017), in which Burgers or related equations can be formulated as one problem among many. While these tools are powerful, they do not provide a ready-made Burgers-turbulence workflow with stochastic forcing, SGS closures, DNS/LES comparison, and NetCDF diagnostics. The third group consists of published Burgers LES and closure-modeling

studies that develop or evaluate particular closures and discretizations (Li & Wang, 2016; e.g., Love, 1980; Maulik & San, 2018; Subel et al., 2021). These works motivate the need for reusable testbeds, but generally do not present maintained, standalone packages toward that purpose.

PyBurgers differs from these alternatives by being narrow in scope (1D Burgers turbulence, DNS and LES), opinionated about defaults (Fourier collocation, 3/2 dealiasing, adaptive CFL), and pluggable along the axes that matter for research, such as time integrators, spatial operators, and SGS closures. Its contribution is not the novelty of Burgers solvers in general, but the packaging of stochastic forcing, DNS/LES modes, multiple SGS closures, configurable numerics, diagnostics, tests, and documentation into a single Burgers-focused Python package.

Software design

The core design goal is that *the physics and the numerics are independently swappable*. This is enforced by three factory interfaces:

- `TemporalIntegrator.get_integrator(scheme_id, nx)` returns an object implementing a uniform `step(u, dt, compute_rhs, zero_nyquist)` contract. Each integrator advertises its own dissipative stability limit, which the solver uses to compute viscous and hyperviscous CFL bounds. This makes the effective stable time step scheme-aware without requiring scheme-specific code in the main loop.
- `SpatialOperator` selects between FD2, FD4, and spectral derivatives. Hyperviscosity is normalized as $\nu_4 = \pi^4 dx^4 / \lambda_{\text{hyp}}$, where λ_{hyp} is the scheme's maximum modified wavenumber magnitude. This ensures that the Nyquist damping rate and the corresponding time-step constraint are scheme-independent. This is essential when comparing schemes because otherwise a finite-difference simulation dissipates small scales at a different rate than a spectral simulation with the same nominal coefficient, making it potentially hard to attribute any differences in the solution.
- `SGS.get_model(model_id, ...)` returns an SGS closure implementing `compute(u, dudx, tke_sgs, dt)`. New closures can be added by subclassing `SGS` and registering an ID; the LES solver requires no other changes.

Performance is achieved through `pyFFTW` with persistent wisdom caching, real-to-complex transforms, and pre-allocated buffers in a `SpectralWorkspace` that is shared between the derivative, dealiasing, and filter operators. The default 8192-point DNS and 512-point LES configuration runs to t=200 seconds in roughly 28 seconds and 5 seconds, respectively, on a 2023 MacBook Pro. This efficiency makes parameter sweeps practical without HPC resources.

Configuration is JSON with schema validation while output is NetCDF. The package depends only on `numpy`, `pyfftw`, `netCDF4`, `filelock`, and `jsonschema`. The codebase is type-annotated, lints under `ruff`, and ships a `pytest` suite covering temporal and spatial operators, SGS closures, input validation, reproducibility, and DNS/LES integration tests. Basic example Python comparison scripts are also provided to explore model output.

Research impact statement

PyBurgers builds on an earlier MATLAB implementation used in published work on dynamic eddy-viscosity model assessment (Basu, 2009). This Python implementation greatly expands on that code, and makes the Burgers-turbulence workflow easier to install, inspect, extend, and combine with the broader scientific Python ecosystem. This is particularly useful for *a posteriori* testing of new SGS closures because researchers can add a closure while reusing the existing DNS/LES solvers, forcing, diagnostics, and NetCDF output pathway. The project has a public development history beginning in 2017, tagged releases, a change log, documentation, contribution guidelines, tests, and Zenodo-indexed citable releases. The code has been used in

a graduate-level LES course in the Department of Mechanical Engineering at the University of Utah, and is currently used by the author and students at the National Severe Storms Laboratory (NSSL) to help test newly developed SGS schemes. These open-development, classroom-usage, and reproducibility signals support its use as a community-facing research and teaching testbed rather than a one-off analysis script.

AI usage disclosure

Portions of version 2, including the pluggable temporal/spatial factory design, speed optimizations, and code documentation, were prepared with the assistance of Anthropic's Claude Code. All algorithmic choices, scientific claims, validation results, and final wording were reviewed and edited by the author, who takes full responsibility for the content.

Acknowledgements

The author thanks Dr. Sukanta Basu for the original MATLAB implementation on which PyBurgers is based, Dr. Rob Stoll for guidance on the formulation and for the use of PyBurgers in his graduate course, and Dr. Louis Wicker for several helpful conversations that informed the direction of the code.

References

Bashforth, F., & Adams, J. C. (1883). *An attempt to test the theories of capillary action*. Cambridge University Press.

Basu, S. (2009). Can the dynamic eddy-viscosity class of subgrid-scale models capture inertial-range properties of burgers turbulence? *Journal of Turbulence*, *10*(12), 1–16. https://doi.org/10.1080/14685240902852719

Bec, J., & Khanin, K. (2007). Burgers turbulence. *Physics Reports*, *447*(1), 1–66. https://doi.org/10.1016/j.physrep.2007.04.002

Binder, S. (2021). *Burgers' equation simulation 1D (simulation de l'équation de burgers)*. https://github.com/sachabinder/Burgers_equation_simulation

Burgers, J. M. (1939). Mathematical examples illustrating relations occurring in the theory of turbulent fluid motion. *Verhandelingen Der Koninklijke Akademie van Wetenschappen, Afdeeling Natuurkunde*, *17, No. 2*, 1–53.

Burns, K. J., Vasil, G. M., Oishi, J. S., Lecoanet, D., & Brown, B. P. (2020). Dedalus: A flexible framework for numerical simulations with spectral methods. *Physical Review Research*, *2*(2), 023068. https://doi.org/10.1103/PhysRevResearch.2.023068

Cantwell, C. D., Moxey, D., Comerford, A., & others. (2015). Nektar++: An open-source spectral/hp element framework. *Computer Physics Communications*, *192*, 205–219. https://doi.org/10.1016/j.cpc.2015.02.008

Deardorff, J. W. (1980). Stratocumulus-capped mixed layers derived from a three-dimensional model. *Boundary-Layer Meteorology*, *18*, 495–527. https://doi.org/10.1007/BF00119502

Durran, D. R. (2010). *Numerical methods for fluid dynamics: With applications to geophysics* (2nd ed., Vol. 32). Springer-Verlag. https://doi.org/10.1007/978-1-4419-6412-0

Germano, M., Piomelli, U., Moin, P., & Cabot, W. H. (1991). A dynamic subgrid-scale eddy viscosity model. *Physics of Fluids A: Fluid Dynamics*, *3*(7), 1760–1765. https://doi.org/10.1063/1.857955

LaBryer, A., Attar, P. J., & Vedula, P. (2015). A framework for large eddy simulation of Burgers turbulence based upon spatial and temporal statistical information. *Physics of Fluids*, *27*(3), 035116. https://doi.org/10.1063/1.4916132

Li, Y., & Wang, Z. J. (2016). A priori and a posteriori evaluations of sub-grid scale models for the Burgers' equation. *Computers & Fluids*, *139*, 92–104. https://doi.org/10.1016/j.compfluid.2016.04.015

Love, M. D. (1980). Subgrid modelling studies with Burgers' equation. *Journal of Fluid Mechanics*, *100*(1), 87–110. https://doi.org/10.1017/S0022112080001024

Maulik, R., & San, O. (2018). Explicit and implicit LES closures for Burgers turbulence. *Journal of Computational and Applied Mathematics*, *327*, 12–40. https://doi.org/10.1016/j.cam.2017.06.003

Moulton, F. R. (1926). *New methods in exterior ballistics*. University of Chicago Press.

Rackauckas, C., & Nie, Q. (2017). DifferentialEquations.jl – a performant and feature-rich ecosystem for solving differential equations in Julia. *Journal of Open Research Software*, *5*(1), 15. https://doi.org/10.5334/jors.151

Smagorinsky, J. (1963). General circulation experiments with the primitive equations: I. The basic experiment. *Monthly Weather Review*, *91*, 99–164. https://doi.org/10.1175/1520-0493(1963)091%3C0099:GCEWTP%3E2.3.CO;2

Subel, A., Chattopadhyay, A., Guan, Y., & Hassanzadeh, P. (2021). Data-driven subgrid-scale modeling of forced Burgers turbulence using deep learning with generalization to higher Reynolds numbers via transfer learning. *Physics of Fluids*, *33*(3), 031702. https://doi.org/10.1063/5.0040286

Williamson, J. H. (1980). Low-storage Runge–Kutta schemes. *Journal of Computational Physics*, *35*(1), 48–56. https://doi.org/10.1016/0021-9991(80)90033-9

Wong, V. C., & Lilly, D. K. (1994). A comparison of two dynamic subgrid closure methods for turbulent thermal convection. *Physics of Fluids*, *6*(2), 1016–1023. https://doi.org/10.1063/1.868335